Intuition Core Upgrade

1 Executive Summary
2 Scope
3 System Overview
4 Security Specification
4.1 Governance and Upgrade Authority
4.2 Curve Fee Hooks and `DynamicFeeFlatPriceCurve`
4.3 Batched Calls and `MultiVaultLib`
4.4 Atom Wallet Ownership
4.5 `FeeProxy`
5 Findings
5.1 A Position Holding the Minimum Eligible Stake Keeps a Tier's Whole Fee Share Once the Vault Moves Above It Major ✓ Fixed
5.2 Affiliate Fee Recipient Runs Before the Routed MultiVault Call Major ✓ Fixed
5.3 Sentinel Accounts Allow Near-Free Repositioning and Advantageous Fee Capture Major ✓ Fixed
5.4 Deposit Approvals Let the Approved Account Move the Receiver's Tier on `DynamicFeeFlatPriceCurve` Medium ✓ Fixed
5.5 Fee Recovery Depends on How a Position Is Split Medium Acknowledged
5.6 Nothing Prevents a Hook-Bearing Curve From Being Set as the Default Curve Minor ✓ Fixed
Appendix 1 - Files in Scope
Appendix 2 - Disclosure
A.2.1 Purpose of Reports
A.2.2 Links to Other Web Sites from This Web Site
A.2.3 Timeliness of Content

Date	September 2026
Auditors	Arturo Roura, George Kobakhidze

1 Executive Summary

This report presents the results of our engagement with **Intuition** to review the **v1.1.0 core upgrade** of the Intuition Protocol smart contracts.

The Intuition Protocol lets users stake `TRUST` to signal confidence in "terms". An atom is the simplest kind of term and can represent anything; three atoms can be combined into a triple, which expresses a subject, predicate, and object relationship and is a term itself. All deposits are held by the `MultiVault` contract, and each term can have several vaults whose share prices follow different bonding curves. More details about the system can be found in the documentation.

We have previously reviewed the `MultiVault` system and its bonding curves, as well as the `TRUST` emissions and bonding contracts. This engagement covers v1.1.0, a release that upgrades several of these contracts at once. Its largest part by volume is a refactor: to stay within the contract size limit, most of `MultiVault`'s logic was moved into a new library, `MultiVaultLib`, which executes on `MultiVault`'s storage. `MultiVault` also gained a payable `multicall` that combines several actions, including ones that send `TRUST`, into a single transaction, and a number of smaller fixes and configuration options were rolled in alongside it. Beyond the vault, the atom wallets and their warden were reimplemented: `AtomWallet` is now built on smart-contract-wallet libraries derived from the Coinbase Smart Wallet and follows a multi-owner model, while `AtomWarden` gained new ways to hand a wallet over to its owner, including claims approved by a group of off-chain signers. A new periphery contract, `FeeProxy`, lets third-party applications route deposits and term creation to `MultiVault` while charging a fee of their own, supporting referral and social-trading use cases. The most consequential change, however, is the new `DynamicFeeFlatPriceCurve`, which keeps the share price flat, charges its fees according to a ladder of vault size tiers, and redistributes them among the vault's existing holders. Supporting it also changed how `MultiVault` works with curves: until now curves only calculated prices, whereas a curve can now charge its own fee, hold it, and keep its own accounting. The remaining changes, such as the new timelock on `MultiVault` settings and the extended pause coverage in `TrustBonding`, are summarized in the System Overview.

With the exception of the curve, these changes are largely self-contained: each is confined to its own module, and none of them alters what it means to hold a position in a vault. The curve is different, because it changes how users behave. Since its share price is flat, all of its economics sit in the fees: a vault's total stake is divided into tiers, the rate charged depends on the tier a deposit lands in or the tier the redeeming holder belongs to, and fees are redistributed to the holders of particular tiers rather than to the vault as a whole. What a position earns therefore depends on where the vault's total stands relative to the tier its holder was assigned to. This encourages repositioning of the kind seen in concentrated-liquidity AMMs, except that, because the share price does not move, positions are rebalanced around the vault's total value rather than around a price.

We did not identify a way for one user to take another user's deposited principal. The fee distribution does favor the most active and best-positioned participants over those who simply hold, but that is a design choice rather than a flaw in itself. Our concern is the complexity behind it: the mechanism admits a wide range of vault states, and each state opens strategies that are difficult to enumerate in advance. Several of the issues in this report are examples of exactly that: states in which fees end up somewhere the design did not intend. As is usually the case, they sit at the edges, where the mechanism starts to diverge from the assumptions behind it, and in a system of this kind it is hard to be confident that every such edge has been found.

The code itself is of good quality, and logic is mostly factored into small, self-contained helpers. The difficulty lies not in any single mechanism, since most are straightforward in isolation, but in how they combine. Calculating a single redemption fee, for example, passes through a multi-tier, multi-recipient split in which nearly every step has a conditional case that zeroes out and sends the amount elsewhere. The resulting cognitive complexity is high: even with the mechanism fully specified, establishing where a given fee ends up takes real effort, which is what makes the behavioral question above so hard to settle.

The update also carries a significant body of tests and makes greater use of established, battle-tested libraries and patterns. Together with the documentation kept alongside the code, this reflects a team that is deliberate about the decisions it makes, both when implementing well-known mechanisms and when designing its own.

During the final week of the review, the Intuition team delivered two further changes to the curve, which we took into the engagement. The first splits the parameters that control how a fee is spread across tiers into separate sets for deposits and for redemptions. The kernel behind the spread is unchanged, so this widens what can be configured rather than how the distribution works. The second changes no contract code: it replaces the schedule the deploy script ships with, adjusting the tier ladder and the fee rates and, more importantly, enabling two distribution routes that were previously switched off. Because this curve's behavior is set as much by its configuration as by its code, that second change alters how fees will flow at launch, and with it the practical weight of some of the issues in this report.

NOTE: After our review, the Intuition team informed us of a further change to the deployment configuration, with no change to contract code. At launch, `DynamicFeeFlatPriceCurve` will be owned directly by the Admin Safe (4-of-8) rather than by the Parameters Timelock, so that the launch parameters can be tuned without a 3-day delay once real deposits arrive. Ownership is planned to move to a timelock later through `Ownable2Step`. The fee ceilings remain immutable and upgrades of the curve remain behind the 7-day Upgrades Timelock, but until ownership is transferred, changes to the curve's fee schedules and distribution parameters take effect immediately.

2 Scope

We use analytics cookies to understand how visitors use our site. You can accept or decline.Learn more
DECLINE ACCEPT

The scope covered the contracts that carry meaningful changes in this release. Contracts affected only by formatting or import changes were excluded, as were interfaces, deployment scripts, and tests, although these were consulted for context.

Our review focused on the revision above, and the file list in the Appendix is taken at it. During the final week of the engagement the Intuition team provided a further set of changes to the curve, up to revision `2fd2c224158e36376d559949f126fd1017686c9a`, which we took into the engagement as well. They are described at the end of the Executive Summary.

Following the review, the Intuition team addressed the findings in this report, and we reviewed their fixes as part of the remediation phase. The final revision reviewed, which includes all of these fixes, is the head of PR #158, `ea6c23985a3d6f12e1e6b22b29836b3c450ac852`. The resolution of each finding names the commit in which it was addressed.

The detailed list of files in scope can be found in the Appendix.

3 System Overview

The v1.1.0 upgrade modifies most of the core Intuition contracts, but the majority of these modifications are small. This overview concentrates on the parts that introduce new behavior: the curve fee hooks and `DynamicFeeFlatPriceCurve`, batching in `MultiVault`, the atom wallet claim process, and `FeeProxy`. The remaining changes are summarized at the end.

Curve fee hooks and `DynamicFeeFlatPriceCurve`

In previous versions, bonding curves only calculated share prices, while `MultiVault` held all assets and did all the accounting. This upgrade lets a curve charge a fee of its own on deposits and redemptions. A curve signals this through two flags, one for each direction, which are off by default, so existing curves are unaffected. When a flag is on, `MultiVault` asks the curve for the fee, withholds it from the user, and, after updating its own balances, sends the fee to the curve together with the details of the deposit or redemption. The curve uses these calls to keep its own record of each user's position, which must always match the user's shares in `MultiVault`.

`DynamicFeeFlatPriceCurve` is the first curve to use these hooks. Its share price is fixed at 1:1, exactly like `LinearCurve`, and all of its logic is in the fees:

- **Tiers.** Each vault's total stake is divided into consecutive tiers whose widths grow geometrically. The width of the first tier, the rate at which the tiers widen, and their number are all configurable. Every holder is also assigned a tier: the average of the tiers their deposits landed in, weighted by amount. It is set when depositing and does not change when redeeming.
- **Fee rates.** The deposit fee rate rises with the tier the vault is in. A deposit that spans several tiers pays each tier's rate on the part that falls into it, and is processed tier by tier, as if it were a sequence of smaller deposits. The redemption fee rate depends on the holder's tier. Both schedules are configurable through a base rate, a per-tier increase, and a cap, and individual tiers can be given their own rates.
- **Fee distribution.** A deposit fee is shared among the holders of the tiers below the one where it was charged. How far down it reaches, and how it is weighted across those tiers, is configurable; tiers closer to the peak of that weighting receive a larger share. Tiers without holders are skipped and their share is spread over the remaining ones; if no tier in range has holders, the whole fee goes to the occupied tier closest to that peak. A redemption fee goes to the other holders of the redeeming holder's tier, and a configurable part of it can instead be spread over the lower tiers in the same way as a deposit fee. If the redeeming holder's tier has no other holders, the whole amount goes to the nearest occupied tier, searching the tiers above first. Within a tier, fees are shared in proportion to stake. Fees with no possible recipient, such as those charged while the vault is still in its first tier, go to a protocol balance that the curve owner can withdraw.
- **Claims.** Fees are not sent to holders directly. The curve records them per tier using a standard fee-per-share accumulator, and holders withdraw their earnings with `claim`. The curve therefore holds the fees that have not yet been claimed or withdrawn.
- **Configuration.** The owner can change the fee schedules and the distribution parameters, override the rates of individual tiers, and require a minimum stake for a tier to receive fees. All rates stay bounded by immutable ceilings in the contract, and several of these features are switched off entirely when their parameter is set to zero.

NOTE: The description above reflects the curve as reviewed. As part of the remediation of issues 5.1 and 5.3, the per-account average tier was replaced by a lot model: an account holds a separate lot in each tier its deposits entered through, a deposit opens or tops up a lot in every tier it crosses without moving existing lots, and a redemption draws from the account's highest lots first. In addition, each tier's share of a distributed fee is now weighted by how full the tier is relative to its width, and whatever thinly filled tiers cannot take goes to the protocol balance. The resolutions of 5.1 and 5.3 describe these changes in detail.

Batching and `MultiVaultLib`

`MultiVault` has a new `multicall` function that runs several of its own functions in a single transaction, including functions that require sending `TRUST`, such as creating an atom and then depositing into it. The caller states how much of the sent `TRUST` each call may use, and these amounts must add up to the total sent. While the batch runs, each call only sees its own amount, which is kept in transient storage, and functions that should not receive `TRUST`, such as `redeem`, reject any non-zero amount. Every call keeps the original sender, and batches cannot be nested.

With these and other additions, `MultiVault` had outgrown the contract size limit. Most of its logic was therefore moved, largely unchanged, into the external library `MultiVaultLib`. `MultiVault` keeps the access control and pause checks and forwards each call to the library, which runs on `MultiVault`'s storage through `delegatecall`. For this to work, the library's description of the storage layout must exactly match `MultiVault`'s.

Atom wallet claims: `AtomWarden` and `AtomWallet`

Each atom has its own smart contract wallet, which collects part of the fees paid on deposits into that atom. Until the wallet is claimed, its ownership rests with the protocol through `AtomWarden`. This release rebuilds how a wallet is handed over. A wallet can now be claimed:

- by the address an atom represents, if the atom is an address, as before;
- by a user holding a claim approved by several of the protocol's off-chain signers. A minimum number of signatures is required, each approval can be used only once and within a time window, and the number of such claims per period can be capped;
- by the atom's creator, once a waiting period has passed and the wallet has collected a minimum amount of fees. To support this, `AtomWarden` now records each atom's creator and creation time.

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

A claim now completes in a single step. From then on, the wallet follows the multi-owner model of the Coinbase Smart Wallet, implemented in `CoinbaseSmartWalletLib`: the owner can add further owners, including passkeys, and any owner can operate the wallet and sign on its behalf. A wallet that has not been claimed cannot be operated.

`FeeProxy`

`FeeProxy` is a new contract that lets third-party applications, called affiliates, route deposits and term creation to `MultiVault` and charge a fee for it. Affiliates register with the proxy and set their own fees within limits set by the protocol. For each action, the proxy deducts the affiliate's fee, forwards the rest to `MultiVault` on the user's behalf, and returns any excess `TRUST`. Since the user remains the depositor or creator of record, the user must first approve `FeeProxy` through `MultiVault`'s approval mechanism.

Other changes

The remaining changes are smaller in scope:

- `MultiVault` settings can now only be changed through a timelock, and pausing is assigned to a separate role.
- Users can allow other accounts to create terms on their behalf, similar to the existing approvals for deposits and redemptions.
- Atoms can be created together with a list of URIs, which are only emitted in an event and do not affect the atom's identity.
- System utilization now carries correctly across epochs, including when an epoch passes without activity, and deposits into counter-triple vaults on non-default curves were fixed.
- `TrustBonding` can now also pause the functions that lock `TRUST`, while withdrawals remain available, and calculates APY values more precisely. `VotingEscrow` was adjusted to make this possible.

4 Security Specification

This section states the properties the mechanisms introduced in v1.1.0 are expected to hold, the roles that can influence them, and the assumptions our review rested on. The parts of the system that this release does not change are covered by our previous reports.

4.1 Governance and Upgrade Authority

On Intuition Mainnet, a single 4-of-8 Gnosis Safe, referred to below as the Admin Safe, holds the privileged roles, either directly or through two timelocks. Through the Upgrades Timelock, which enforces a 7-day delay, it can replace the implementations of `MultiVault`, `TrustBonding`, `AtomWarden`, all atom wallets, and `DynamicFeeFlatPriceCurve`. The Parameters Timelock, which enforces a 3-day delay, is the only account that can change `MultiVault`'s configuration. At the time of our review it was also to be the owner of the curve; following a later change to the deployment configuration, the Admin Safe will instead own `DynamicFeeFlatPriceCurve` directly at launch, with ownership planned to move to a timelock later through `Ownable2Step`. Until then, the owner's changes to the curve's configuration take effect without delay. `FeeProxy` has its own admin and its own upgrade authority, both set when it is deployed.

Everything below assumes these role holders act as intended and that their keys are not compromised. Each of them can change or replace the mechanisms described here, and an upgrade can alter any logic or storage, including user balances.

This release must also be rolled out as a single atomic batch. The upgraded vault queries the fee-hook getters on the curve of every deposit and redemption, and curve implementations deployed before this release do not expose them.

4.2 Curve Fee Hooks and `DynamicFeeFlatPriceCurve`

Expected behavior. `MultiVault` asks a fee-charging curve for its fee, withholds exactly that amount from the user, and forwards it to the curve when reporting the deposit or redemption; the quoted amount and the forwarded amount are the same. The curve's own record of each position matches the user's share balance in `MultiVault` at all times. Every fee it receives is either credited to the holders of a tier or recorded as protocol accrual, apart from sub-wei rounding remainders that stay in the contract, and the curve holds enough `TRUST` to pay everything it has credited. A holder never earns from the fee charged on their own redemption. Earnings already recorded stay claimable and are not affected by later configuration changes.

Roles. Only `MultiVault`, whose address is fixed when the curve is initialized, can report deposits and redemptions. The owner sets the tier layout, both fee schedules, the distribution parameters, per-tier rate overrides, and the minimum stake a tier needs to receive fees, and can withdraw the accrued protocol fees to any address; all rates remain bounded by immutable ceilings in the contract. The registry owner decides which curves exist at all, and registration cannot be undone.

Trust assumptions. Fees are redistributed between users, so who earns what depends on the configuration in force and on the state of the vault at the time; participants are expected to position themselves to earn more. A curve that charges fees must never be set as the default curve, because term creation does not call the hooks; this constraint is documented but not enforced in code. For the same reason, any future path that changes share balances without calling the hooks would break the correspondence between the two ledgers.

4.3 Batched Calls and `MultiVaultLib`

Expected behavior. A batch moves exactly the value sent with it: the per-call allocations must add up to `msg.value`, each call sees only its own allocation, and no call can spend value allocated to another. Entry points that do not take value reject a non-zero allocation, both inside a batch and on a direct call. Batches cannot be nested, and every call executes with the original sender and on `MultiVault`'s storage. The library operates on that same storage through an identical layout, and `MultiVault` applies its pause and access-control checks before forwarding.

Roles. None are added by this mechanism.

Trust assumptions. The value accounting holds only while every payable entry point either consumes just its allocation or rejects value outright; this is recorded in the code as a rule for future changes. The layouts of the vault and the library must likewise be kept identical in every future upgrade.

4.4 Atom Wallet Ownership

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

be operated. A signed claim requires signatures from a minimum number of distinct accounts holding `SIGNER_ROLE` over an authorization that names the claimant, the atom, a nonce, and a validity window; it can be used only by that claimant, only once, and only within that window, and the number of such claims can be capped per time window. The creator fallback is available only to the atom's recorded creator, only after the configured waiting period has passed, and only once the wallet has accumulated at least the configured amount of fees. After a claim, only the wallet's registered owners can manage its signers, and only they or the ERC-4337 EntryPoint can execute transactions through it.

Roles. The Admin Safe holds `DEFAULT_ADMIN_ROLE`, which manages roles, sets every claim parameter, can pause the contract, and can repoint it at a different `MultiVault`. `OPERATOR_ROLE` can assign any unclaimed wallet to any address and can invalidate pending authorizations. `SIGNER_ROLE` is held by the keys that approve claims. Once a wallet is claimed, its owners fully control it and the fees it has collected, and can add further owners or rotate the primary owner.

Trust assumptions. The infrastructure that decides which claims to authorize, and the custody of the signing keys, are off-chain and outside this review. The per-window cap bounds how much a compromise of those keys can reach at once, but does not prevent it. Adding an owner to a claimed wallet grants that owner the same powers as the one who added it.

4.5 `FeeProxy`

Expected behavior. For each routed action, the proxy deducts the affiliate's fee, forwards the remainder to `MultiVault`, and attributes the deposit or the term creation to the user rather than to itself. The fee must be within the protocol's caps and within the maximum the caller states on that call. Excess value is returned to the caller, and when that transfer fails it is credited to them to withdraw later. If the call to `MultiVault` fails, the whole transaction, including the affiliate payment, is reverted.

Roles. The admin sets the fee caps and the registration fee, manages roles, and is the only role that can lift a pause, whether global or for a single affiliate; `PAUSER_ROLE` can apply either pause. Affiliates set their own fees within the caps and may change them at any time.

Trust assumptions. Affiliates are not trusted: callers are protected by the protocol's caps and by the maximum fee they pass with each call. A user who approves `FeeProxy` in `MultiVault` allows it to create terms and deposit in their name for as long as that approval stands.

5 Findings

Each issue has an assigned severity:

- Critical issues are directly exploitable security vulnerabilities that need to be fixed.
- Major issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Medium issues are objective in nature but are not security vulnerabilities. These should be addressed unless there is a clear reason not to.
- Minor issues are subjective in nature. They are typically suggestions around best practices or readability. Code maintainers should use their own judgment as to whether to address such issues.
- Issues without a severity are general recommendations or optional improvements. They are not related to security or correctness and may be addressed at the discretion of the maintainer.

5.1 A Position Holding the Minimum Eligible Stake Keeps a Tier's Whole Fee Share Once the Vault Moves Above It Major ✓ Fixed

Resolution

Fixed in 6c83910 (0xIntuition/intuition-contracts-v2@`6c83910`. `userTier` is replaced by per-tier lots, so a redeem removes stake from the tiers it actually sits in and no tier standing is left behind. Each tier's share of the fee is now weighted by its fill `(tierStake / width)`, and the lump and reroutes are capped the same way. A 1-wei tier earns a 1-wei share, and the rest goes to the protocol bucket, so the attack earns nothing back.

Description

A holder's tier is written only on deposit. `recordRedeem` reduces `userStake` and `tierStake` but not `userTier`, so a holder can redeem almost all of a position and keep its tier.

A tier's share of each fee does not depend on its stake. `_weighPriorTiers` weights tiers by distance from the fulcrum only, and the slice is then divided by the tier's own stake. Any tier whose stake meets `minEligibleTierStake` is paid in full, so a sole occupant holding exactly that amount earns what the tier would earn holding 10,000 TRUST.

Nobody new can join a tier once the vault reads above it: deposits land in the vault's current band, and existing holders only move toward it.

A depositor can therefore fill a tier while the vault is still in it. Other users' deposits then move the vault past that tier, and the depositor redeems everything except `minEligibleTierStake`. The redemption lowers the vault again, but as long as other users' stake keeps it above the tier, the position keeps the tier's whole share of deposit fees and of the down-ladder part of exit fees with its capital withdrawn. issue 5.3 does the same across several accounts; this needs one.

The only limit is `minEligibleTierStake`, which is checked against the tier's total stake. The attacker has to keep that amount in the tier rather than 1 wei. It is 0 at launch, so any non-zero balance qualifies, and it cannot be set above 1,000 TRUST. At any value the attack still works; a higher floor only increases the capital left in the tier.

Examples

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L896-L922

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

```
uint256 newAvg;
if (oldStake == 0) {
    newAvg = bandTier * TIER_PRECISION;
} else {
    // Unrolled into per-term 512-bit-intermediate mulDivs so the weighted average cannot
    // overflow for any stake magnitude (the naive sum-of-products form already had ~99 bits
    // of headroom at max tier x total native supply, but this removes the question
    // permanently). The two floors under-round the average by at most 2 wei-tier units
    // (2e-18 of a tier) per band — far below the half-up bucket-rounding threshold.
    newAvg = userAvgTier[termId][account].fullMulDiv(oldStake, newStake)
        + (bandTier * TIER_PRECISION).fullMulDiv(bandStake, newStake);
}
uint256 newTier = _roundTier(newAvg);

if (newTier != oldTier) {
    if (oldStake > 0) {
        tierStake[termId][oldTier] -= oldStake;
    }
    tierStake[termId][newTier] += newStake;
} else {
    tierStake[termId][newTier] += bandStake;
}

userStake[termId][account] = newStake;
userAvgTier[termId][account] = newAvg;
userTier[termId][account] = newTier;
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L496-L503

```
uint256 exitTier = userTier[termId][account];

// Settle the exiter, then remove the exiting portion so they are excluded from the fee they
// are about to pay.
_settle(termId, account);
userStake[termId][account] -= shares;
tierStake[termId][exitTier] -= shares;
uint256 residual = userStake[termId][account];
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L1097-L1105

```
if (!_isEligibleStake(recipientStake)) {
    recipientStake = 0;
}
stakes[d - 1] = recipientStake;
if (recipientStake > 0) {
    uint256 w = _triangularWeight(_fulcrumDistance(d, dStar), sigma);
    weights[d - 1] = w;
    sumWeights += w;
}
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L1063

```
accFeePerShare[termId][span - d] += slice.fullMulDiv(ACC_PRECISION, stakes[d - 1]);
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L974-L976

```
function _isEligibleStake(uint256 stake) private view returns (bool) {
    return stake > 0 && stake >= config.minEligibleTierStake;
}
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L137

```
uint256 public constant MAX_MIN_ELIGIBLE_TIER_STAKE = 1000e18;
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L942-L962

```
function _payDepositFee(bytes32 termId, uint256 feeAmount, uint256 sourceTier) private {
    if (feeAmount == 0) return;

    uint256 toPriorTier = feeAmount.mulDiv(config.depositToPriorTierBps, BPS);
    uint256 toFulcrum = feeAmount - toPriorTier;

    // Prior-tier spike -> the single nearest eligible prior tier. If none qualifies, fold the
    // slice into the fulcrum pool rather than forfeiting it.
    if (toPriorTier > 0) {
        (uint256 recipientTier, uint256 recipientStake) = _nearestEligiblePriorTier(termId, sourceTier);
        // Sentinel, not an occupancy test: the scan above already applies the floor.
        if (recipientStake > 0) {
            accFeePerShare[termId][recipientTier] += toPriorTier.fullMulDiv(ACC_PRECISION, recipientStake);
        } else {
            toFulcrum += toPriorTier;
        }
    }

    // Fulcrum slice (+ any un-spikable remainder) -> the prior tiers by the triangular kernel.
    _payFulcrumTiers(termId, toFulcrum, sourceTier, 0, 0);
}
```

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

```
new market, vault 0
attacker deposits 2,000           ->  fills tier 0, userTier 0, vault 2,000 (reads tier 1)
other users deposit 4,000         ->  held at tier 1, vault 6,000 (reads tier 2)
attacker redeems all but 1 wei    ->  userTier stays 0, vault 4,000 (reads tier 1)

tier 0   attacker       1 wei
tier 1   other users    4,000
```

With the vault at tier 1, every deposit lands in band 1, and its fee can only reach tier 0: the kernel has no other prior tier, and the 10% lump goes to the nearest occupied tier below, which is also tier 0. The attacker's 1 wei takes the whole fee and the 4,000 in tier 1 takes nothing. New deposits land in tier 1 and cannot join tier 0.

As later deposits move the vault up and later depositors occupy the tiers they deposit into, tier 0 keeps a share close to tier 1's while holding 1 wei against 4,000:

vault reads	deposit fee	tier 0 (1 wei)	tier 1 (4,000)	tier 0 share of exit fees
tier 1	1.45%	100.0%	0.0%	75.0%
tier 2	1.90%	40.9%	59.1%	34.1%
tier 3	2.35%	26.1%	31.9%	21.8%
tier 4	2.80%	18.0%	22.5%	15.0%
tier 5	3.25%	13.4%	17.2%	11.2%

The round trip costs about 137 (MultiVault fees in and out, the tier 0 deposit fee and the tier 0 redeem fee). Deposit fees alone recover it after about 23,800 of new deposits, by which point the vault reads tier 5; exit fees shorten that.

Recommendation

Recompute the tier for the remaining stake on redemption, or base eligibility on each position's lowest stake since its last deposit. Consider weighting a tier's share by the stake it holds. Raising `minEligibleTierStake` only raises the amount the attacker leaves in the tier.

5.2 Affiliate Fee Recipient Runs Before the Routed MultiVault Call Major ✓ Fixed

Resolution

Fixed in commit 0dc6c1e by placing the `_payAffiliate()` execution after the `MultiVault` calls return.

Description

Every `FeeProxy` routing entry point pays the affiliate before it calls `MultiVault`. The payment is done via `Address.sendValue`, which simply performs `.call` operation to the affiliate's `feeRecipient` with all remaining gas. So whenever the affiliate fee is non-zero the recipient runs arbitrary code after the user's parameters are fixed and before the user's operation executes.

Affiliates register permissionlessly and can change `feeRecipient` at any time through `updateFeeRecipient`. `FeeProxy`'s `nonReentrant` does not cover `MultiVault`, so the recipient can deposit, redeem and create on `MultiVault` directly. On a chain with a single sequencer and no public mempool, this is a guaranteed position ahead of every user routing through the affiliate.

The recipient can change any state the user's operation is priced from:

- **Price on progressive curves.** `OffsetProgressiveCurve` prices each share at `0.1 * (totalShares + 30)` TRUST as deployed. A deposit from the callback raises the price the user pays, the user's deposit raises it further, and the recipient redeems in the next transaction. The only bound is the user's `minShares`, which the recipient can push the user down to; with `minShares` at 0 there is none.
- **Fee predicates.** `_shouldChargeFees` and `_shouldChargeAtomDepositFraction` compare default-vault shares against `feeThreshold`. The recipient can deposit to flip them, so the user pays a fee their simulation did not include into a vault the recipient now holds. `createAtomsFor` and `createTriplesFor` take no output floor.

Examples

The recipient is called before `MultiVault`; the other four routing entry points use the same order:

src/periphery/FeeProxy.sol:L273-L275

```
_payAffiliate(row.feeRecipient, affiliate, msg.sender, affiliateFee);
shares = IMultiVault(multiVault).deposit{ value: forwardedAssets }(receiver, termId, curveId, minShares);
```

src/periphery/FeeProxy.sol:L744-L751

```
function _payAffiliate(address feeRecipient, address affiliate, address user, uint256 affiliateFee) internal {
    if (affiliateFee == 0) {
        emit AffiliateFeePaid(affiliate, user, 0);
        return;
    }
    Address.sendValue(payable(feeRecipient), affiliateFee);
    emit AffiliateFeePaid(affiliate, user, affiliateFee);
}
```

src/periphery/FeeProxy.sol:L234-L244

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

```
    if (feeRecipient == address(0)) revert FeeProxy_ZeroAddress();

    AffiliateConfig storage row = _affiliateConfigs[msg.sender];
    if (row.registeredAt == 0) revert FeeProxy_AffiliateNotRegistered(msg.sender);

    address previous = row.feeRecipient;
    row.feeRecipient = feeRecipient;

    emit AffiliateFeeRecipientUpdated(msg.sender, previous, feeRecipient);
}
```

Recommendation

Move `_payAffiliate` after the `MultiVault` call on all five routing entry points, or accrue affiliate fees for withdrawal instead of pushing them.

5.3 Sentinel Accounts Allow Near-Free Repositioning and Advantageous Fee Capture Major ✓ Fixed

Resolution

Fixed in 6c83910 (0xIntuition/intuition-contracts-v2@`6c83910`, which addresses both halves of the strategy:

- The per-account average tier is gone. Stake is now held in lots, one per account per tier, each fixed to the band it entered through. A deposit opens or tops up a lot in each band it crosses and never touches existing lots. A redeem takes from the highest lots first. Replaying Phase 2: the 10,529 deposit creates lots at tiers 3 and 4, and the 10,529 withdrawal drains exactly those lots. The 8,345 stays at tiers 0–2. Stake can no longer be registered at a tier it didn't enter through.
- The kernel spread now multiplies each tier's weight by its fill `(stake / width)` and caps the common per-share rate at the schedule. A 1-wei seat therefore earns a 1-wei share rather than a whole tier's slice. The deposit spike and the exit-fee reroutes are capped the same way, at each recipient's fill. Whatever thin tiers can't take goes to the protocol bucket, not to the seat holders. The exiter's remaining lots are excluded on every tier during a redeem.

Description

Component: `DynamicFeeFlatPriceCurve` **Config:** launch values as provided (`width0 2000e18`, `tierWidthGrowthBps 3500`, `fulcrumAlphaBps 5000`, `kernelSpread 6e18`, `depositToPriorTierBps 1000`, `redeemToFulcrumTiersBps 7500`, `minEligibleTierStake 0`)

Three separately minor behaviours combine into one cheap, repeatable strategy:

1. A holder's tier is a single per-account average, so multiple tiers require multiple accounts
2. Redemptions never recompute `userTier` or `userAvgTier`
3. Deposit and redeem fees are paid to prior tiers and to the exiting tier's cohort

Individually these are documented and intended. Together they let one actor hold every tier below the frontier through sentinel accounts, move their entire stack up to the frontier with a single deposit and withdrawal, and recover all of the curve fees paid for that move.

The only curve fee in the strategy that can never be recovered is on the first deposit into tier 0, which has no prior tier to receive it and goes to the protocol, 20.00 in this example. Beyond that, the only real cost is MultiVault's own fees, 3.0% of the amount moved (3.8% of the stack).

The resulting position keeps earning from tiers no one else can ever enter, dilutes genuine depositors at the tier where the stack sits, and makes every later repositioning cheaper for the attacker than for any other participant.

Tier ladder at launch config

tier	width	upper edge	deposit fee	redeem fee
0	2,000	2,000	1.00%	1.50%
1	2,700	4,700	1.45%	1.85%
2	3,645	8,345	1.90%	2.20%
3	4,921	13,266	2.35%	2.55%
4	6,643	19,909	2.80%	2.90%
5	8,968	28,877	3.25%	3.25%

Deposit cap (5.5%) binds from tier 10, redeem cap (4.5%) from tier 9.

Phase 1: fill each tier, leave a sentinel behind

As the vault grows, a main account fills each tier. A separate account is left holding a minimal position (1 wei in the simulation) in each. Separate accounts are required because one account only ever occupies one tier bucket.

```
deposit 2,000 at band 0  ->  stake 2,000   avg 0.0000   userTier 0
deposit 2,700 at band 1  ->  stake 4,700   avg 0.5745   userTier 1
deposit 3,645 at band 2  ->  stake 8,345   avg 1.1971   userTier 1
```

Main account holds 8,345 registered at tier 1. Sentinels sit at `t0`, `t1`, `t2`. The vault total sits exactly on `edge(2)`, and since `_tierOf` uses a strict `<`, it already reads as tier 3, so the next deposit lands in band 3.

The tier 0 deposit fee is the one that cannot be recaptured

A deposit charged at tier 0 has `span = 0`. There is no prior tier, so `_payFulcrumTiers` sends the entire fee to `protocolAccrued`. No sentinel arrangement can capture it, by construction.

```
band 0:  2,000 x 1.00% =  20.00   ->  protocolAccrued                  unrecoverable
band 1:  2,700 x 1.45% =  39.15   ->  t0 (main account, S0)            recaptured
band 2:  3,645 x 1.90% =  69.26   ->  t1, t0 (main account, S1, S0)    recaptured
```

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

come straight back. The tier 0 deposit is the only exception in the entire strategy, and it is paid once.

Phase 2: one deposit, one withdrawal

A deposit raises the weighted average. A withdrawal returns the capital and leaves the average untouched, since `recordRedeem` never writes `userTier` or `userAvgTier`. So depositing and then withdrawing the same amount leaves the committed stake unchanged but registered at a higher tier.

The cheapest version is a single deposit large enough to land the average on 2.5, followed by withdrawing that same amount. The minimum is 10,528.83; the walkthrough uses 10,529 so the average clears 2.5 under integer rounding.

```
deposit 10,529
  band 3:  4,920.75  ->  avg 1.8659   userTier 2
  band 4:  5,608.25  ->  avg 2.5000   userTier 3
  vault 18,874, reads tier 4

withdraw 10,529
  stake back to 8,345, avg stays 2.5000, userTier 3
  vault 8,345, reads tier 3, band 3 physically empty
```

The whole 8,345 is now registered at tier 3, and band 3 is empty for genuine depositors to fill, where they land against the stack.

Why the curve fees come back

The 100% figures below refer to the Phase 2 repositioning. The tier 0 deposit fee from Phase 1 is the only curve fee in the strategy that cannot be recaptured, as covered above.

Deposit leg: 100% recaptured

The deposit is replayed band by band, and each band's fee is paid to the tiers below it before that band's stake lands.

```
band 3:  4,920.75 x 2.35% = 115.64      (main account at tier 1)
   lump   11.56 -> t2
   kernel span 3:  t2 36.93 | t1 36.93 | t0 30.21

band 4:  5,608.25 x 2.80% = 157.03      (main account at tier 2)
   lump   15.70 -> t2
   kernel span 4:  t2 56.53 | t1 47.11 | t0 37.69     (t3 empty, renormalized away)
```

Every recipient tier is attacker-held. The main account also collects directly, since it is itself registered in one of the recipient tiers while each band's fee is paid: tier 1 during band 3, tier 2 during band 4.

Redeem leg: 100% recaptured

Withdrawing 10,529 at tier 3 costs 2.55% = 268.49. With `redeemToFulcrumTiersBps = 7500`, 201.37 flows down-ladder through the fulcrum and 67.12 goes to the exiting tier's cohort.

The 75% fulcrum portion. The vault reads tier 4 at withdrawal, so `span = 4` and the window reaches tier 3. Tier 3 holds only the main account's residual, which is excluded, so it is zeroed and renormalized away.

```
201.37  ->  t2 80.55 | t1 67.12 | t0 53.70      (t3 renormalized away)
```

The 25% exiting-tier portion.

```
cohortStake = tierStake[exitTier] - residual;
```

The main account is alone at tier 3, so excluding its residual leaves `cohortStake = 0`. That triggers the reroute: `_nearestEligibleTier` searches upward first (tiers 4 to 15, all empty), then downward, and lands on the t2 sentinel.

Result

```
Phase 2 curve fees paid    deposit 272.67 + redeem 268.49 = 541.16
  to attacker accounts                                       541.16   (100%)
  to protocolAccrued                                         4 wei
```

The two real deterrents

With the Phase 2 curve fees fully recaptured, these are the only costs.

1. MultiVault fees

Charged before the curve hook runs, so they sit outside the recapture loop entirely. Using deploy-script values (protocol 125bps, entry 50, exit 75, atom wallet 50), on the 10,529 moved:

```
protocol fee, deposit       131.61    unrecoverable
atom wallet fee, deposit     52.65    unrecoverable
protocol fee, redeem        131.61    unrecoverable
entry + exit fees           131.61    pro-rata to vault holders, mostly recoverable if dominant

hard leak 315.87, 3.0% of the amount moved
```

These MultiVault rates were not among the launch values provided, so they are assumed unchanged from the deploy script and should be confirmed.

2. The tier 0 deposit fee

Covered in Phase 1: 20.00, unrecoverable because `span = 0` leaves no prior tier to pay. A genuine deterrent, but one-time and negligible, since every later deposit lands at a tier that has prior tiers beneath it.

Net effect on the intended deterrent

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

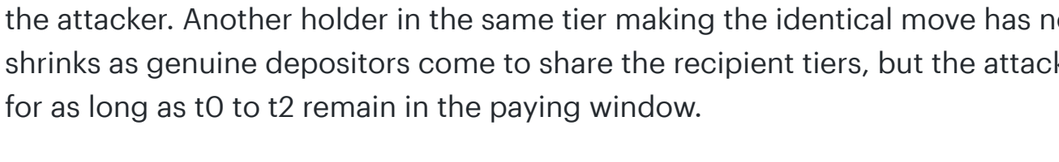
```
MultiVault hard leak                                net   315.87
                                                          --------
                                                  total     315.87

= 3.0% of the 10,529 moved, 3.8% of the 8,345 stack, paid once
plus 20.00 tier 0 deposit fee
```

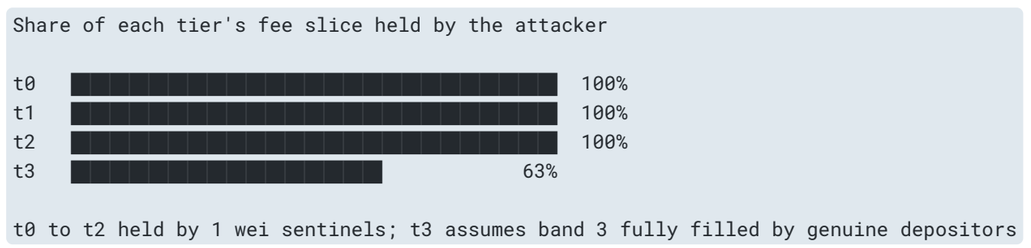
The design relies on the deposit and redeem spread to make repositioning unattractive. That reasoning assumes the repositioner and the cohort receiving the fee are different parties. For an actor holding the tiers below through sentinels they are the same party. The curve's own fees on the move net to zero, and the entire remaining cost is MultiVault's fees.

The position this creates

After Phase 2 the whole 8,345 is registered at tier 3, yet none of it sits in band 3, which is still physically empty. Genuine depositors can only ever put 4,921 into band 3.



Once genuine depositors fill band 3, the attacker holds 63% of tier 3. The sentinels, meanwhile, stay the only occupants of t0 to t2, since no fresh deposit can land in a tier the vault has already passed.



This covers deposit fees and, with `redeemToFulcrumTiersBps = 7500`, three quarters of every exit fee as well. Honest stake at t0 to t2 would collect the same slices; the sentinels do it with one wei each, so the same capital is also earning at tier 3. The t0 to t2 slices shrink as the vault grows, and t0 leaves the paying window at vault tier 12.

Later repositioning stays cheaper than for anyone else

The same deposit and withdrawal can be repeated as the vault grows, pushing the stack toward whichever tier pays most. Each move's fees are split across the tiers below, where the sentinels still sit, so part of every deposit and withdrawal fee comes back to the attacker. Another holder in the same tier making the identical move has no sentinels below and recovers far less. The return shrinks as genuine depositors come to share the recipient tiers, but the attacker's repositioning stays cheaper than anyone else's for as long as t0 to t2 remain in the paying window.

The launch config makes this easier, not harder

The kernel change delivered in the final week widens the window of tiers that share each fee, so a sentinel keeps earning well after the vault has climbed past it instead of falling out of range a few tiers later. A tier's share does not depend on how much stake sits in it, so every tier added to that window is one more slot worth holding for the price of a minimum deposit. The deposit lump and the share of the exit fee now routed through the same window add further slices to those same slots. The more tiers the curve pays at once, the better the return on capturing each one, and the launch values pay more of them, for longer.

5.4 Deposit Approvals Let the Approved Account Move the Receiver's Tier on `DynamicFeeFlatPriceCurve` Medium ✓ Fixed

Resolution
Addressed by documentation in commit 2d0e6ed by noting the behavior in the code comments, and addressed structurally in commit 6c83910.

Description

`MultiVault` lets an account deposit on behalf of another account once the receiver has granted it the `DEPOSIT` approval. On most curves this only adds shares to the receiver's balance. On `DynamicFeeFlatPriceCurve` the deposit is recorded against the receiver's position, and it changes where that position sits.

`_processDeposit` passes `receiver` to the curve's deposit hook. `_applyDepositBand` then recomputes the receiver's stake-weighted average tier from their existing stake and the new stake at the vault's current band, and rewrites `userTier`. The receiver's tier decides which fees the position receives, since fees are paid only to tiers below the source band, and which redeem rate the position pays, since `quoteRedeemFee` keys on `userTier`.

An approved account therefore controls more than the amount deposited. It can move the receiver's position toward the vault's current tier, out of the tiers that are being paid and into a higher redeem rate. The amount required depends on the receiver's existing stake. A small position moves almost entirely for a trivial deposit, which matters for positions such as the one described in issue 5.1.

The receiver cannot undo the change. Redemptions do not recompute the tier, and any deposit the receiver makes lands at the current band as well, so the position cannot return to its previous tier until the vault itself falls back below it. The approved account pays for the deposit and the receiver keeps the resulting shares, so no funds are taken from the receiver; the position loses the fee share it was earning and pays a higher exit fee.

The `ApprovalTypes` documentation describes `DEPOSIT` only as a bit flag and does not mention that it grants control over curve position.

Examples

We use analytics cookies to understand how visitors use our site. You can accept or decline.

We use analytics cookies to understand how visitors use our site. You can accept or decline.Learn more

src/libraries/MultiVaultLib.sol:L264-L266

```
if (!_isApprovedToDeposit(msg.sender, receiver)) {
    revert MultiVault.MultiVault_SenderNotApproved();
}
```

src/libraries/MultiVaultLib.sol:L1498-L1500

```
function _isApprovedToDeposit(address sender, address receiver) private view returns (bool) {
    return sender == receiver || (_s().approvals[receiver][sender] & uint8(ApprovalTypes.DEPOSIT)) != 0;
}
```

The curve hook is called with the receiver:

src/libraries/MultiVaultLib.sol:L838

```
_recordCurveDeposit(termId, receiver, hook, sharesForReceiver);
```

The receiver's average and tier are rewritten from the new band:

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L896-L922

```
uint256 newStake = oldStake + bandStake;
uint256 newAvg;
if (oldStake == 0) {
    newAvg = bandTier * TIER_PRECISION;
} else {
    // Unrolled into per-term 512-bit-intermediate mulDivs so the weighted average cannot
    // overflow for any stake magnitude (the naive sum-of-products form already had ~99 bits
    // of headroom at max tier x total native supply, but this removes the question
    // permanently). The two floors under-round the average by at most 2 wei-tier units
    // (2e-18 of a tier) per band — far below the half-up bucket-rounding threshold.
    newAvg = userAvgTier[termId][account].fullMulDiv(oldStake, newStake)
        + (bandTier * TIER_PRECISION).fullMulDiv(bandStake, newStake);
}
uint256 newTier = _roundTier(newAvg);

if (newTier != oldTier) {
    if (oldStake > 0) {
        tierStake[termId][oldTier] -= oldStake;
    }
    tierStake[termId][newTier] += newStake;
} else {
    tierStake[termId][newTier] += bandStake;
}

userStake[termId][account] = newStake;
userAvgTier[termId][account] = newAvg;
userTier[termId][account] = newTier;
```

The redeem rate keys on that tier, and redemptions never rewrite it:

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L423

```
uint256 rateTier = userStake[termId][account] > 0 ? userTier[termId][account] : _tierOf(vaultStake[termId]);
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L496-L503

```
uint256 exitTier = userTier[termId][account];

// Settle the exiter, then remove the exiting portion so they are excluded from the fee they
// are about to pay.
_settle(termId, account);
userStake[termId][account] -= shares;
tierStake[termId][exitTier] -= shares;
uint256 residual = userStake[termId][account];
```

The approval documentation:

src/interfaces/IMultiVault.sol:L31-L46

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

```
/// @dev Values are bit flags packed into the `uint8` slot of
///      `approvals[receiver][sender]`. Bit 0 = DEPOSIT, bit 1 = REDEMPTION,
///      bit 2 = CREATION. Composite values are bitwise unions of the
///      primitives, so a single setter call covers any combination.
///      `BOTH` is kept for backward compatibility and means
///      `DEPOSIT | REDEMPTION` (its historical meaning).
///
///      NONE                    = 0 (0b000)
///      DEPOSIT                 = 1 (0b001)
///      REDEMPTION              = 2 (0b010)
///      BOTH                    = 3 (0b011) = DEPOSIT | REDEMPTION
///      CREATION                = 4 (0b100)
///      DEPOSIT_AND_CREATION    = 5 (0b101) = DEPOSIT | CREATION
///      REDEMPTION_AND_CREATION = 6 (0b110) = REDEMPTION | CREATION
///      ALL                     = 7 (0b111) = DEPOSIT | REDEMPTION | CREATION
```

Launch values as in issue 5.3. The receiver holds a position at tier 1 while the vault reads tier 3, and the approved account deposits at band 3. Deposit fees from band 3 are paid to tiers 0 to 2; tier 3 receives none of them.

receiver stake	approved deposit	receiver avg	userTier	redeem rate
2,700	-	1.00	1	1.85%
2,700	900	1.50	2	2.20%
2,700	8,100	2.50	3	2.55%
1 wei	0.01	3.00	3	2.55%

A position of 2,700 needs 900 to be moved into tier 2 and 8,100 to be moved into tier 3, out of every band 3 deposit fee. A 1 wei position is moved into tier 3 for 0.01.

Recommendation

Document in `ApprovalTypes` and `approve` that on curves with a deposit hook, `DEPOSIT` lets the approved account change the receiver's fee tier and redeem rate, and that the change is permanent. Consider a separate approval type for deposits into curves with a deposit hook, or recording approved deposits without moving the receiver's existing tier.

5.5 Fee Recovery Depends on How a Position Is Split Medium Acknowledged

Resolution

Acknowledged. The Intuition team responded with the following message: "Pro-rata redistribution model cannot distinguish two addresses of one holder; the fix for 5.3 makes the deposit-leg split residual exactly zero, and the redeem-leg recapture stays bounded per share."

Description

The curve stops a position from collecting its own fees in two narrow ways: a redemption excludes only the address that is redeeming, and a deposit's fee is paid out before that deposit's stake is added. Neither check looks at who owns the stake. How much of its own fees a holder gets back therefore depends on how their stake is split across addresses and transactions, not just on how much they hold. An automated holder managing many small positions can pay a lower effective fee than a holder with the same stake in one position.

Redemptions. When an address redeems, only that address's remaining stake is removed from the recipients of the exit fee:

```
// recordRedeem
userStake[termId][account] -= shares;
tierStake[termId][exitTier] -= shares;
uint256 residual = userStake[termId][account];

// exiting-tier share: only this address's residual is excluded
uint256 cohortStake = tierStake[termId][exitTier] - residual;

// down-ladder share: again only this address's residual is excluded
_payFulcrumTiers(termId, toFulcrum + undistributed, currentTier, exitTier, residual);
```

Every other address in the receiving tiers keeps its share, including other accounts that belong to the same holder.

A holder who withdraws a single position gets none of its exit fee back. The same stake spread across many accounts and withdrawn one account at a time behaves differently: each withdrawal's fee is shared with the holder's accounts that have not withdrawn yet, in proportion to their stake. The first withdrawal recovers the most, each later one recovers less, and the last recovers nothing. Splitting within a single account does not help, because that account's remainder is excluded every time.

Deposits. A deposit's fee is paid out before its stake is added, and nobody is excluded from receiving it, including the depositor's existing stake:

```
// _applyDepositBand: the fee is paid out before this deposit's stake is added
_payDepositFee(termId, bandFee, bandTier);
...
userStake[termId][account] = newStake;

// _payDepositFee: nobody is excluded from receiving it
_payFulcrumTiers(termId, toFulcrum, sourceTier, 0, 0);
```

This matters for an account whose tier is one of the tiers receiving the fee, such as an anchor whose average is still climbing through rewarded tiers. Take a deposit that stays within one band. Made in one transaction, its whole fee is paid before any of the new stake lands. Made as many small transactions, the stake arrives a little at a time, so each later piece pays part of its fee to the

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

moves its tier out of the receiving range, the remaining pieces pay nothing back and splitting recovers less than a single deposit would. Whether splitting a deposit helps therefore depends on the pool's configuration and on the amount deposited.

Splitting is effectively rewarded. Holding stake across several accounts in the same tier earns exactly what one position there would, since fees are shared pro-rata by stake, so holding it split costs nothing. The benefit comes on the way out: a split position recovers part of its own exit fee, while a single position recovers none. For an account still climbing through rewarded tiers, splitting deposits can recover more as well. The only cost is extra transactions, which an automated holder absorbs easily. The rational setup for anyone who expects to withdraw is therefore to split, and whatever a split position recovers comes out of what the other holders in those tiers would otherwise have received.

5.6 Nothing Prevents a Hook-Bearing Curve From Being Set as the Default Curve Minor ✓ Fixed

Resolution

Fixed in commit 73a8dcd by adding `_assertDefaultCurveIsHookless()` to `setBondingCurveConfig()` and `initialize()` functions which set default curves via modifying `bondingCurveConfig` that carries the default curve.

Description

`IBaseCurve` states that a curve with fee hooks must not be set as `defaultCurveId`, but `setBondingCurveConfig` does not check it.

`MultiVault` calls a curve's hooks based on the curve id, so with a hook-bearing curve such as `DynamicFeeFlatPriceCurve` as the default, deposits and redemptions on it go through the hooks. `_createAtom` and `_createTriple` do not: they mint the creator's shares and the `minShare` seeds without calling `recordDeposit`. This affects every atom and triple created afterwards.

Created shares cannot be redeemed. `recordRedeem` subtracts them from a `userStake` that never included them and reverts. Resetting `defaultCurveId` does not help, since the hook is resolved from the vault's own curve id.

The curve reads every default vault at the wrong tier. Tier, deposit fee rate and fee recipients come from `vaultStake`, which counts only recorded deposits:

1. Created shares are missing. An atom created with 10,000 TRUST holds about 9,825 shares, tier 3 on the launch ladder, but the curve reads tier 0: deposits are charged 1.00% instead of 2.35%, and the whole fee goes to `protocolAccrued`.
2. The default vault receives entry and exit fees, and the atom deposit fraction, as assets without shares. Its share price rises above 1:1, and `vaultStake`, counted in shares, falls further behind the asset-denominated ladder with every fee. The curve's NatSpec relies on its vault never receiving these fees.

Examples

src/interfaces/IBaseCurve.sol:L153-L160

```
/// @dev    The term-creation paths are the one exception. `createAtoms`,
///         `createAtomsFor`, `createTriples` and `createTriplesFor` mint the creator's shares on
///         `bondingCurveConfig.defaultCurveId` without resolving or dispatching this hook, so a
///         hook-bearing curve would receive no ledger entry for a created position and the
///         holder's first redemption would underflow inside the curve with no recovery path.
///         A hook-bearing curve therefore must not be set as `defaultCurveId`. The deployed
///         configuration keeps `defaultCurveId == 1` (`LinearCurve`, hookless, 1:1); hook curves
///         are reachable only through `deposit` / `redeem` on an explicit non-default `curveId`.
```

src/protocol/MultiVault.sol:L813-L816

```
function setBondingCurveConfig(BondingCurveConfig memory _bondingCurveConfig) external onlyTimelock {
    bondingCurveConfig = _bondingCurveConfig;
    emit BondingCurveConfigUpdated(_bondingCurveConfig.registry, _bondingCurveConfig.defaultCurveId);
}
```

src/libraries/MultiVaultLib.sol:L631-L640

```
uint256 curveId = s.bondingCurveConfig.defaultCurveId;

(uint256 sharesForReceiver, uint256 feeBaseAssets, uint256 assetsAfterFees) =
    _calculateAtomCreate(atomId, assets);

_accumulateVaultProtocolFees(feeBaseAssets);
address atomWallet = _accumulateAtomWalletFees(atomId, feeBaseAssets);

uint256 receiverSharesAfter =
    _updateVaultOnCreation(creator, atomId, curveId, assetsAfterFees, sharesForReceiver, VaultType.ATOM);
```

src/libraries/MultiVaultLib.sol:L717-L733

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

```
(uint256 sharesForReceiver, uint256 feeBaseAssets, uint256 assetsAfterFees) =
    _calculateTripleCreate(tripleId, assets);

_accumulateVaultProtocolFees(feeBaseAssets);

uint256 receiverSharesAfter =
    _updateVaultOnCreation(creator, tripleId, curveId, assetsAfterFees, sharesForReceiver, VaultType.TRIPLE);

if (_shouldChargeAtomDepositFraction(tripleId)) {
    _increaseProRataVaultsAssets(
        tripleId, _feeOnRaw(feeBaseAssets, s.tripleConfig.atomDepositFractionForTriple)
    );
}

_initializeOppositeTripleVault(tripleId, curveId);
```

src/libraries/MultiVaultLib.sol:L907-L918

```
function _depositFeeHookCurve(uint256 curveId) private view returns (address) {
    address curve = IBondingCurveRegistry(_s().bondingCurveConfig.registry).curveAddresses(curveId);
    if (curve == address(0) || !IBaseCurve(curve).hasDepositFeeHook()) return address(0);
    return curve;
}

/// @dev Redeem-path mirror of {_depositFeeHookCurve}, gated on {IBaseCurve.hasRedeemFeeHook}.
function _redeemFeeHookCurve(uint256 curveId) private view returns (address) {
    address curve = IBondingCurveRegistry(_s().bondingCurveConfig.registry).curveAddresses(curveId);
    if (curve == address(0) || !IBaseCurve(curve).hasRedeemFeeHook()) return address(0);
    return curve;
}
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L500-L502

```
_settle(termId, account);
userStake[termId][account] -= shares;
tierStake[termId][exitTier] -= shares;
```

src/protocol/curves/DynamicFeeFlatPriceCurve.sol:L155-L165

```
/// @notice Cumulative net user stake per vault (mirrors the spec's `totalAssets`; excludes the
///         MultiVault min-share seed). Drives the current tier for fee rate + distribution.
/// @dev    Unit coupling, not otherwise asserted. This accumulates share amounts (the record hooks
///         are handed `sharesForReceiver` / `shares`), while the tier ladder it is compared
///         against (`_tierUpperEdge`, `width0`) is denominated in assets (TRUST wei). The two
///         agree only because a vault on this curve holds price at exactly 1:1: MultiVault routes
///         entry and exit fees to the default curve's vault, never to a non-default curve's, so
///         this vault's `totalAssets / totalShares` never drifts off par. Every tier decision, and
///         so every fee rate, depends on that. Crediting pro-rata value to a non-default curve's
///         vault would make the ladder read the wrong quantity.
mapping(bytes32 termId => uint256 stake) public vaultStake;
```

src/libraries/MultiVaultLib.sol:L1174-L1179

```
function _increaseProRataVaultAssets(bytes32 termId, uint256 assets, VaultType vaultType) private {
    Storage storage s = _s();
    uint256 curveId = s.bondingCurveConfig.defaultCurveId;
    VaultState storage vaultState = s.vaults[termId][curveId];
    _setVaultTotals(termId, curveId, vaultState.totalAssets + assets, vaultState.totalShares, vaultType);
}
```

Recommendation

In `setBondingCurveConfig`, revert if the curve at `defaultCurveId` reports `hasDepositFeeHook()` or `hasRedeemFeeHook()`.

Appendix 1 - Files in Scope

This review covered the following files:

File	SHA-1 hash
src/external/curve/VotingEscrow.sol	`609b8645d27453145a9613ea5822b45f0f1d6a86`
src/libraries/CoinbaseSmartWalletLib.sol	`8d0f4fa4aa57b6d3128fda911022634b78c4fd2c`
src/libraries/MultiVaultLib.sol	`0f1d532cd77a788dfe97d40881b31a3bc5889177`
src/periphery/FeeProxy.sol	`30e0cc68ae9f6cd9a4b716298441aabc62f73837`
src/protocol/MultiVault.sol	`b1041a9ec3deac05fbc7249223205b099db96d32`
src/protocol/MultiVaultCore.sol	`be618a3b1d23971409cbf221bc0050e0a680e136`
src/protocol/curves/BaseCurve.sol	`71d46b9104d2d0720cff570a25155167f27ba212`
src/protocol/curves/DynamicFeeFlatPriceCurve.sol	`dbef3c27253b6103e64909aefbd6d604aa28dada`
src/protocol/emissions/TrustBonding.sol	`41cdedf4b8351a2b5a3a5cb545620b2847107863`

We use analytics cookies to understand how visitors use our site. You can accept or decline. Learn more

src/protocol/wallet/AtomWallet.sol	`ea74433bf7cdb54dba8d771e40896fdf00eda8e7`
src/protocol/wallet/AtomWarden.sol	`2d4f1a4ec4e165fa120fd174d87dcb60ae0b844d`

Appendix 2 - Disclosure

Consensys Diligence (“CD”) typically receives compensation from one or more clients (the “Clients”) for performing the analysis contained in these reports (the “Reports”). The Reports may be distributed through other means, including via Consensys publications and other distributions.

The Reports are not an endorsement or indictment of any particular project or team, and the Reports do not guarantee the security of any particular project. This Report does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. No Report provides any warranty or representation to any third party in any respect, including regarding the bug-free nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the Reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset. Specifically, for the avoidance of doubt, this Report does not constitute investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. CD owes no duty to any third party by virtue of publishing these Reports.

A.2.1 Purpose of Reports

The Reports and the analysis described therein are created solely for Clients and published with their consent. The scope of our review is limited to a review of code and only the code we note as being within the scope of our review within this report. Any Solidity code itself presents unique and unquantifiable risks as the Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond specified code that could present security risks. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. In some instances, we may perform penetration testing or infrastructure assessments depending on the scope of the particular engagement.

CD makes the Reports available to parties other than the Clients (i.e., “third parties”) on its website. CD hopes that by making these analyses publicly available, it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.

A.2.2 Links to Other Web Sites from This Web Site

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Consensys and CD. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites’ owners. You agree that Consensys and CD are not responsible for the content or operation of such Web sites, and that Consensys and CD shall have no liability to you or any other person or entity for the use of third party Web sites. Except as described below, a hyperlink from this web Site to another web site does not imply or mean that Consensys and CD endorses the content on that Web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the Reports. Consensys and CD assumes no responsibility for the use of third-party software on the Web Site and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.

A.2.3 Timeliness of Content

The content contained in the Reports is current as of the date appearing on the Report and is subject to change without notice unless indicated otherwise, by Consensys and CD.

2026 All rights reserved